第一次基础实验 实验文档

Merhyi, Last Upd: 2026.09.12

- 第一次基础实验 实验文档
  - 00 实验环境配置
    - Python 的安装与配置（可跳过）
    - VSCode 的安装与配置
    - venv 虚拟环境配置
    - uv 的安装与配置
    - Anaconda 的安装与配置
    - VSCode 选择解释器
  - 01 实验目标与任务
    - 1 计算等额本息还款金额
    - 2 计算三角形的内角
    - 3 内积与二阶矩阵求逆
    - 4 列表与字符串操作
  - 02 验收内容

00 实验环境配置

Python 的安装与配置（可跳过）

在 Windows 环境下，只需进入 Python 的中文下载网站 https://pythonlang.cn/downloads/windows/ 挑选合适的版本下载其安装程序（64-bit，现在应该不会有同学买到 32 位的系统了）

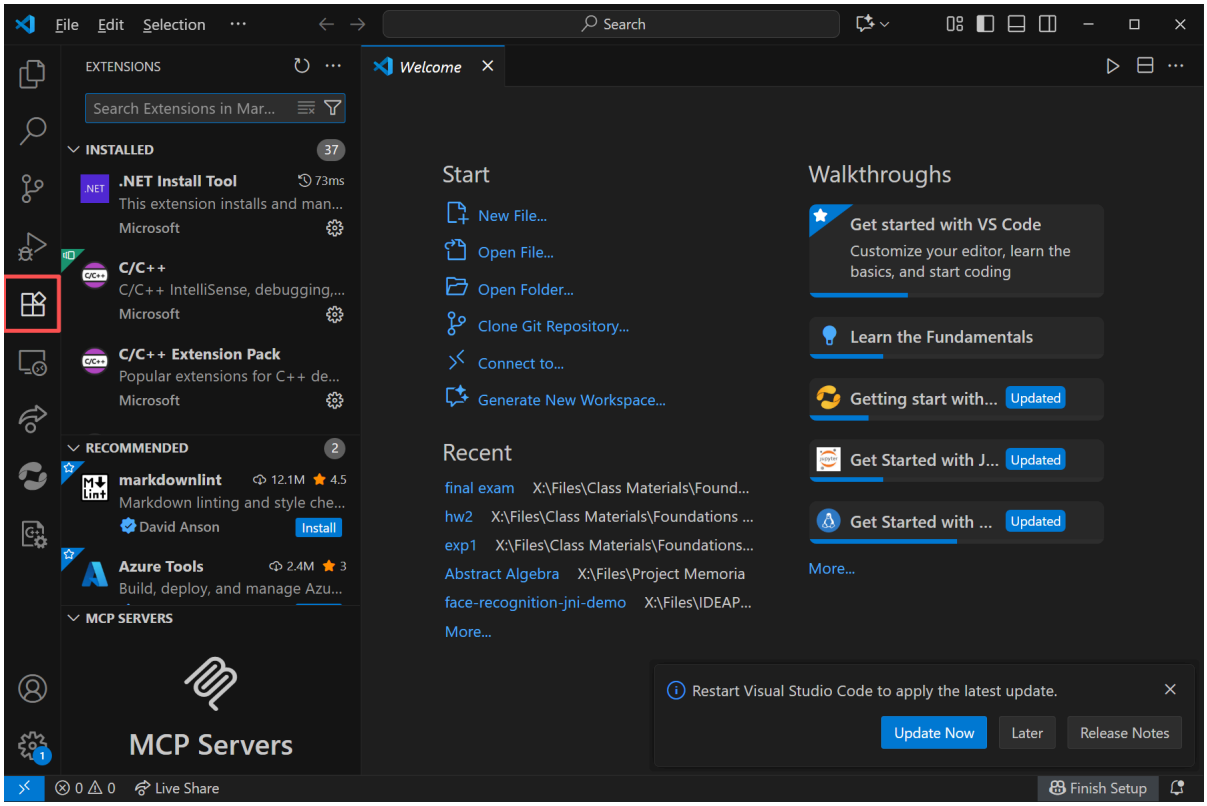
在自己的 Windows 系统上开发的话，建议使用 3.12 左右的版本。如果计划使用 Anaconda 进行环境管理，则实际上不需要为系统手动安装 Python，Anaconda 会根据你所为环境指定的 Python 版本自行维护并下载对应的解释器。

VSCode 的安装与配置

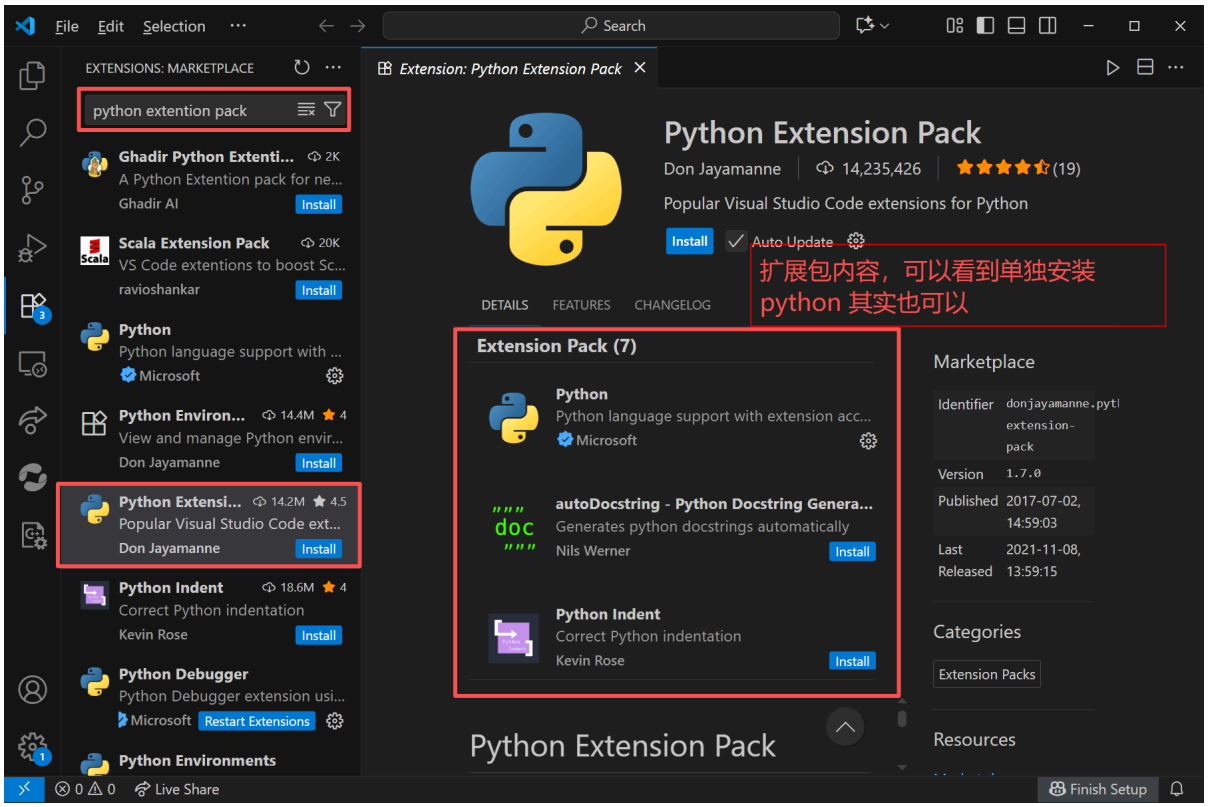
访问 VSCode 的官网 https://code.visualstudio.com/ 下载其安装包后安装即可。

第一次安装 VSCode 后，需要为其配置用于 Python 开发的插件。

进入 VSCode，从左侧边栏进入插件列表

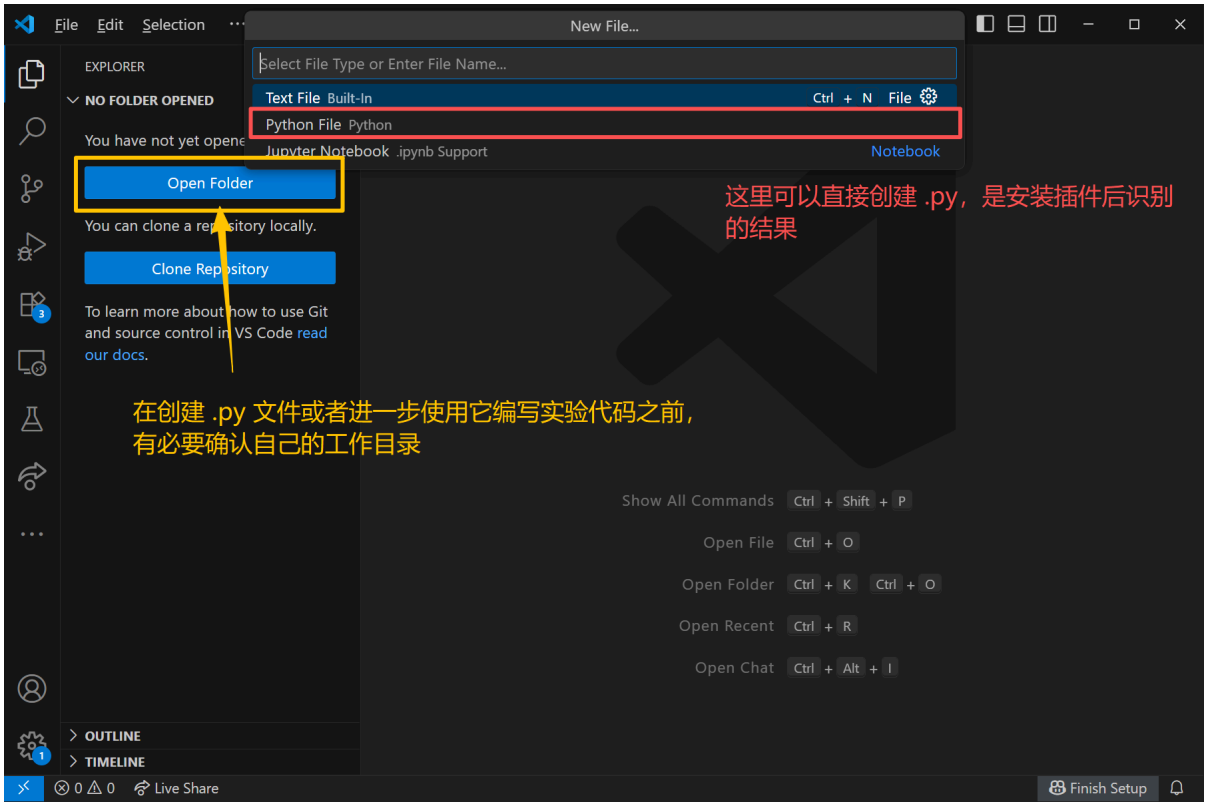


于扩展（EXTENTIONS）处搜索 "python extention pack"，找到相关的插件



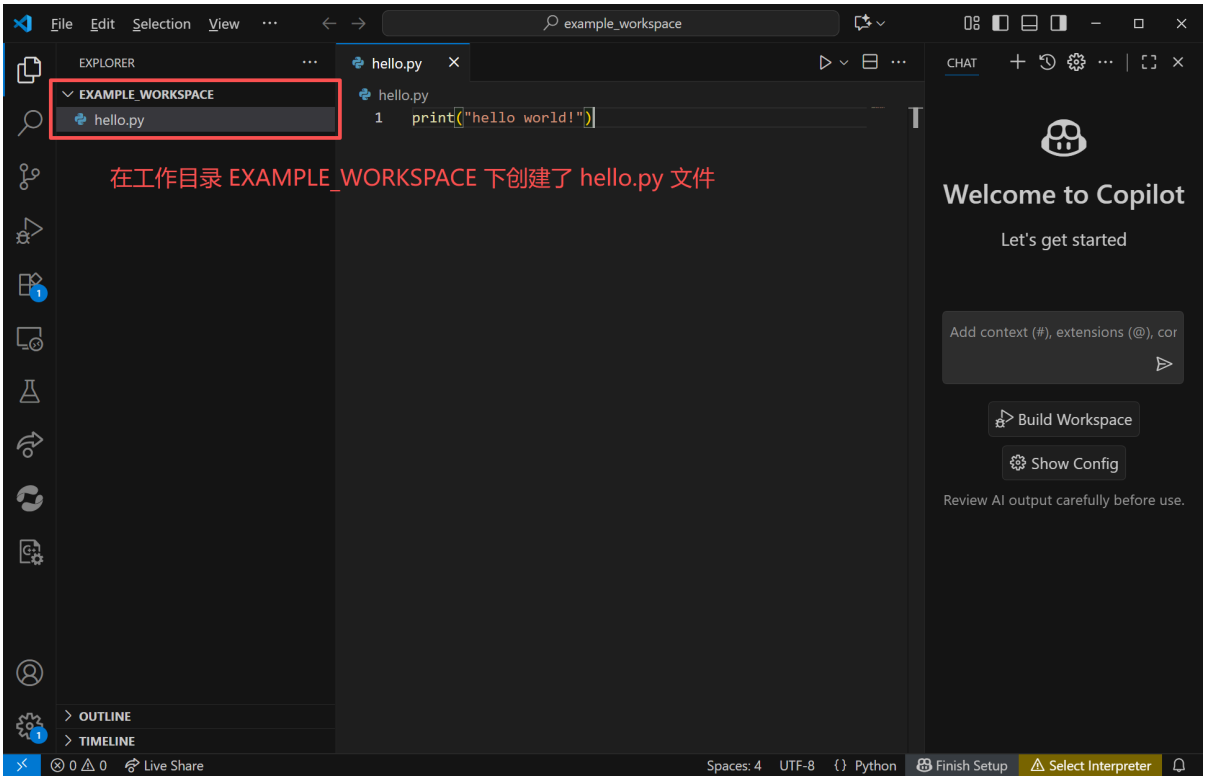
可以自由探索插件库，安装所需要的插件，例如中文插件（Chinese simplified）、代码排版插件等。

在 VSCode 中创建一个新的 `.py` 文件（顶部导航栏处 File -> New File），此时应该能够直接创建 `.py`：

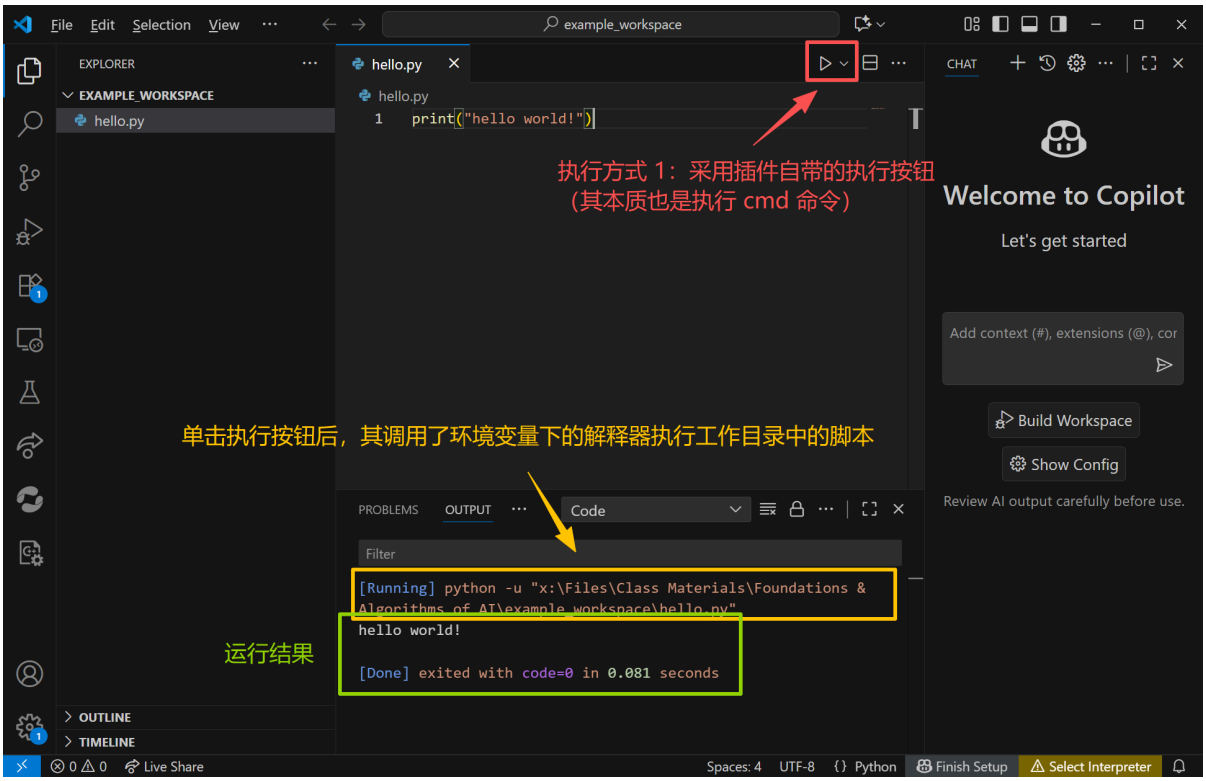


注意这里的 "Open Folder" 选项，其实指定了当前 VSCode 的所有操作将基于的根目录。若在 VSCode 中打开终端，终端的执行位置将会以工作目录为准。在每次创建一个项目之前，先为它规划好指定的目录，并创建文件夹作为工作目录是很有必要的。具体内容参照实验任务。

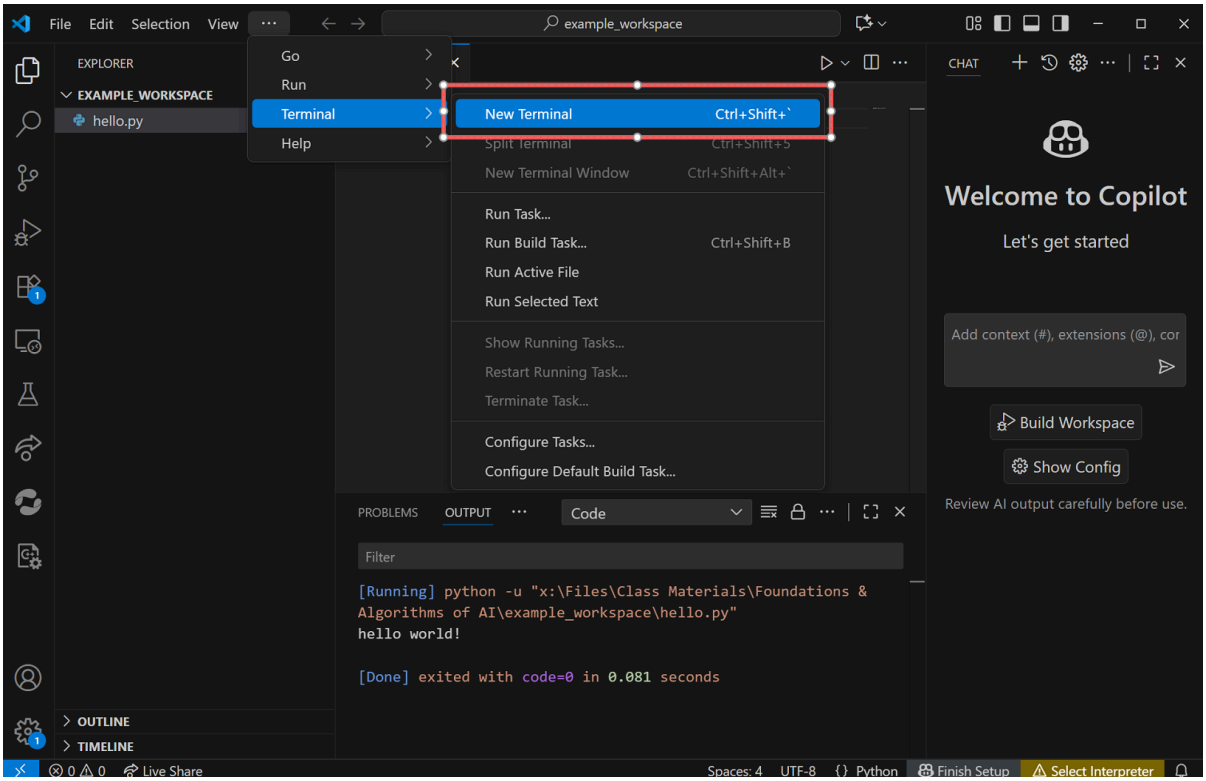
这里，创建工作目录 "example_workspace" 后，在工作目录中新建 `hello.py` 文件，并为其编写经典的打印代码 `hello world!` (^ >ワ<^) .′.′

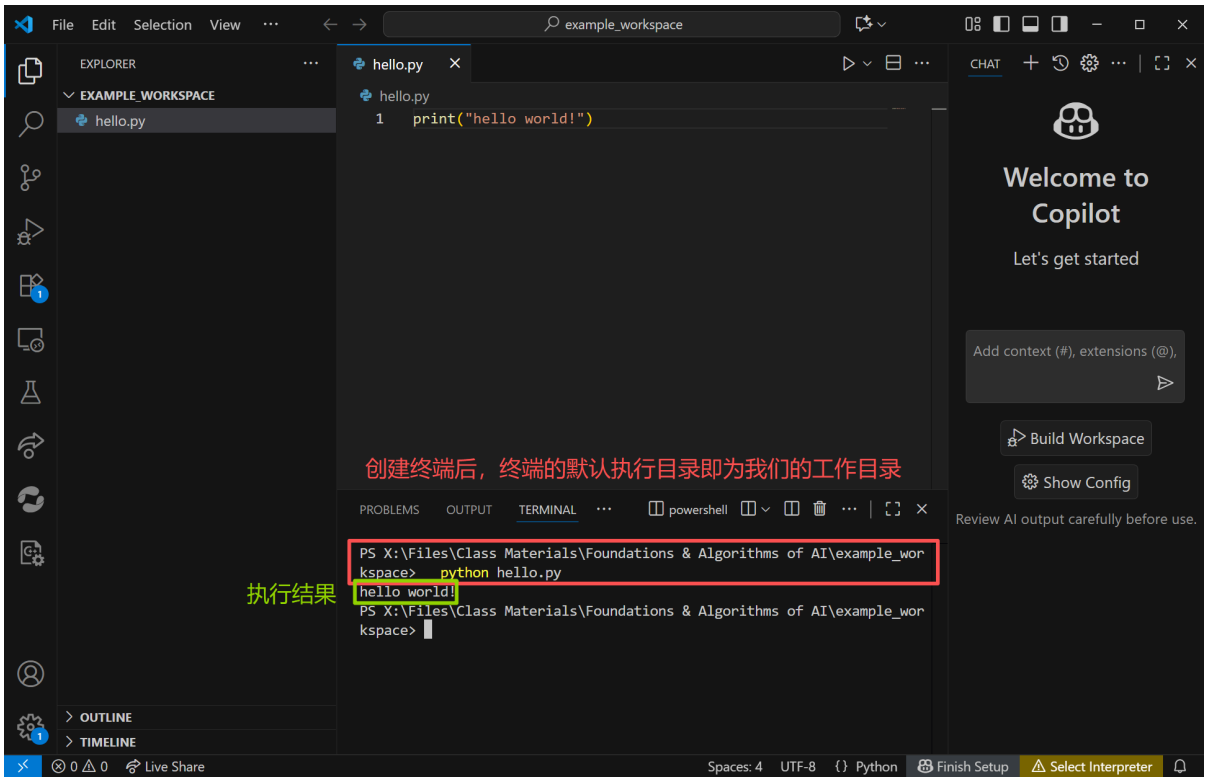


有两种方式执行我们刚才所创建的脚本。方式一，用插件自带的执行功能：



方式二，启动 VSCode 下的终端，手动执行脚本





至此，就能使用 VSCode 进行基本的 Python 开发了。

venv 虚拟环境配置

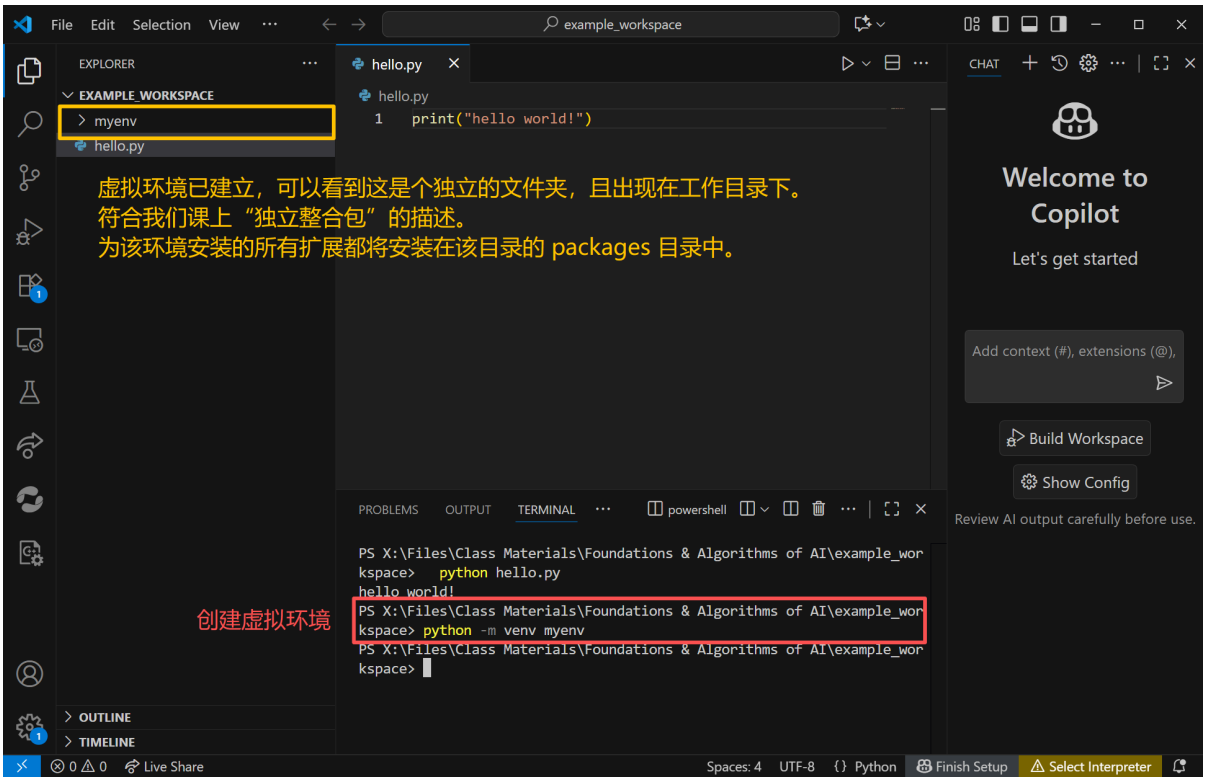
在 Anaconda 无法正常安装或执行的时候，可能不得不使用 venv 进行包管理。此时必须先手动为系统安装 Python，才能在基于系统版本的前提下配置虚拟环境。

在 VSCode 下打开终端，随后为 Python 安装虚拟环境所需要的库：

```
pip install virtualenv
```

接着，通过模块调用它以在工作目录下创建虚拟环境：

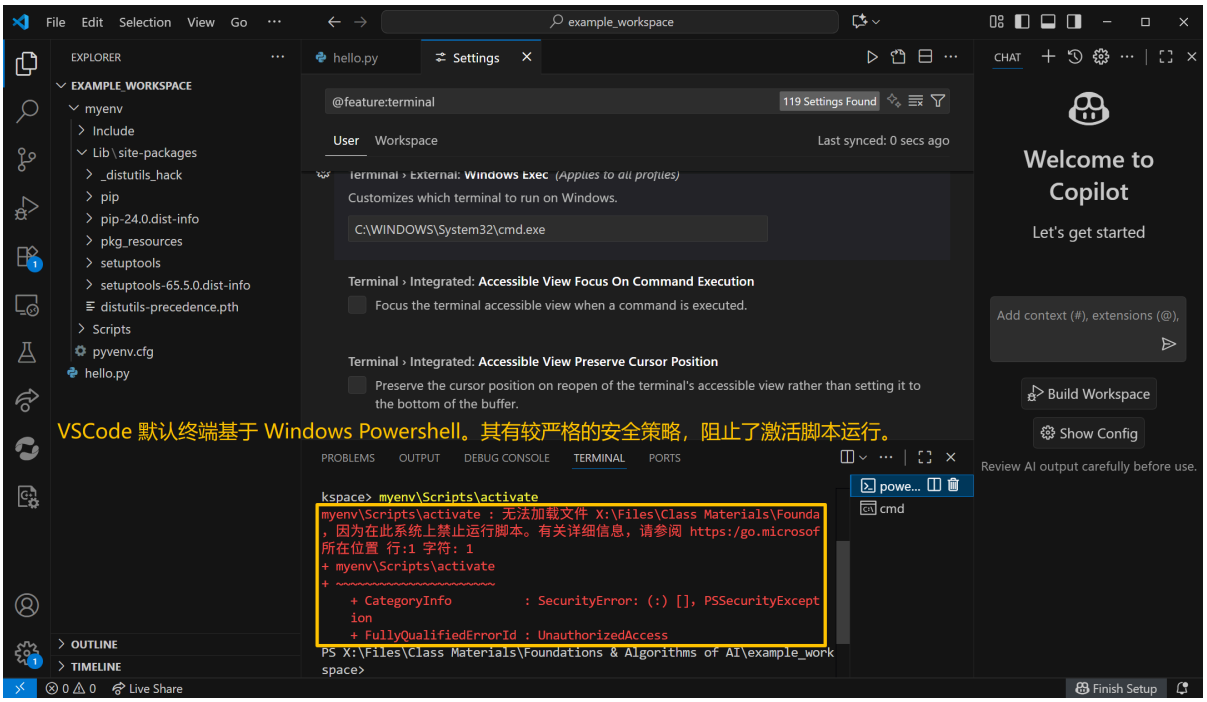
```
python -m venv myenv
```



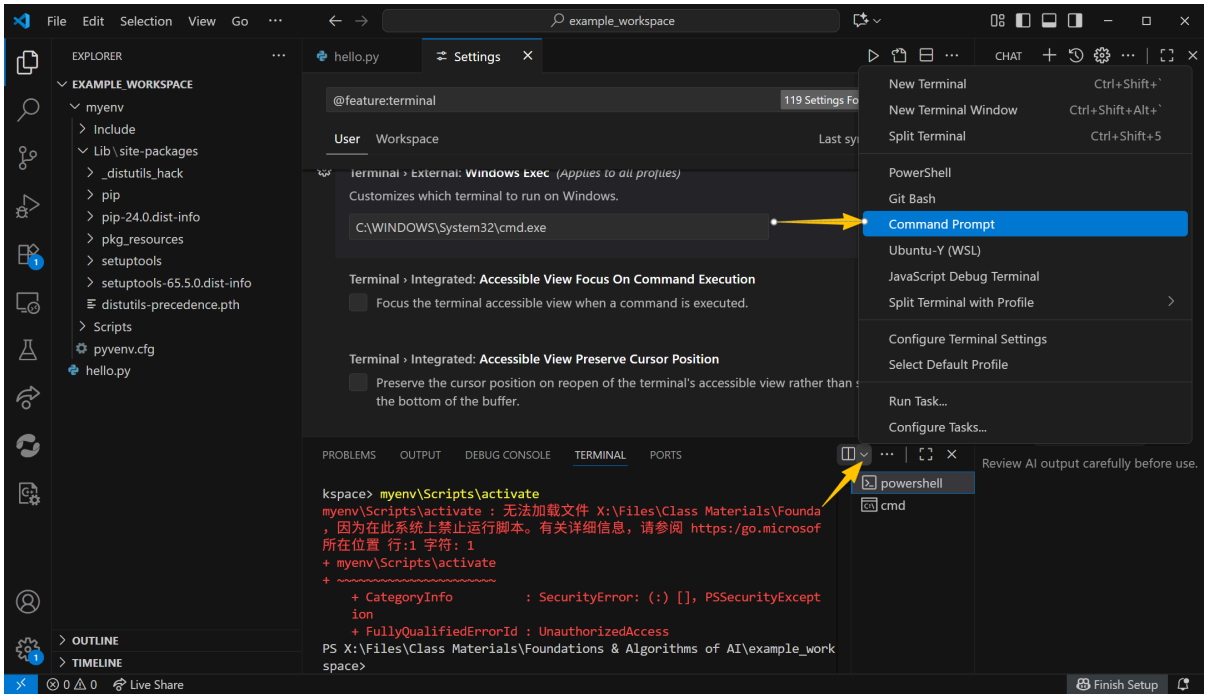
最后，激活该虚拟环境

```
myenv\Scripts\activate
```

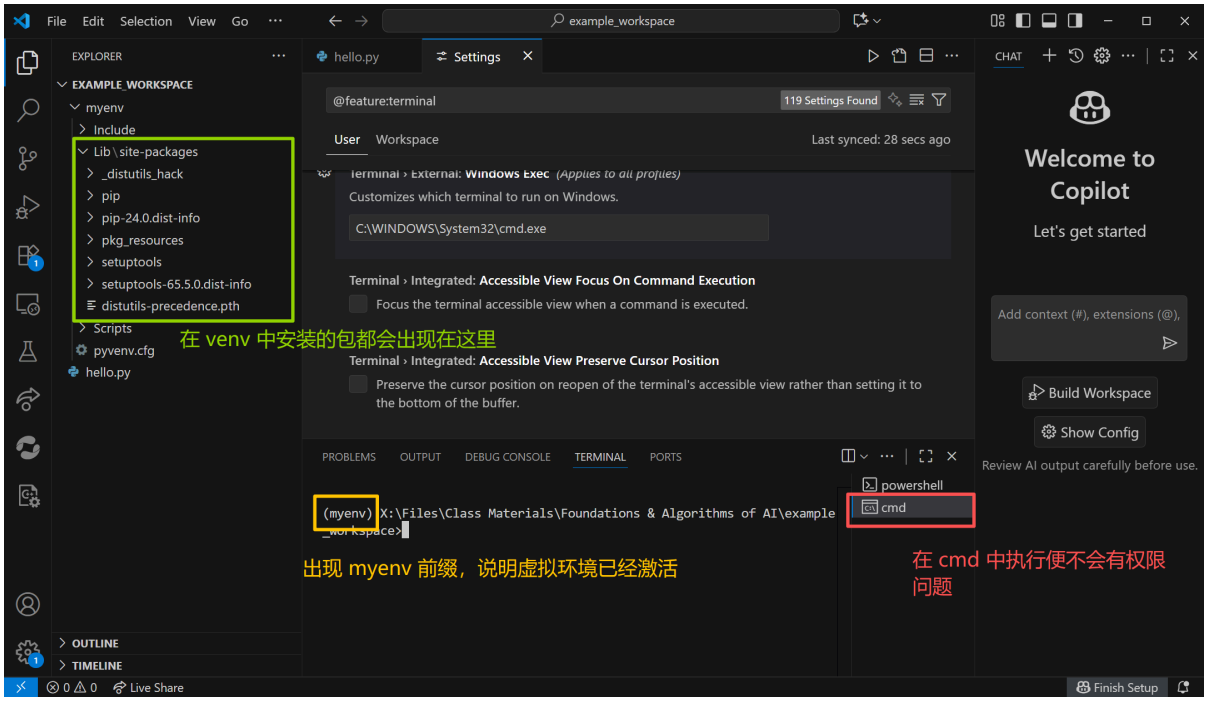
在 VSCode 的默认终端 Powershell 下，会出现报错：



此时需要另外打开一个 cmd 终端：



在 cmd 终端下执行激活命令可正常执行：



uv 的安装与配置

在 VSCode 的 Powershell 终端下，使用 winget 获取 uv

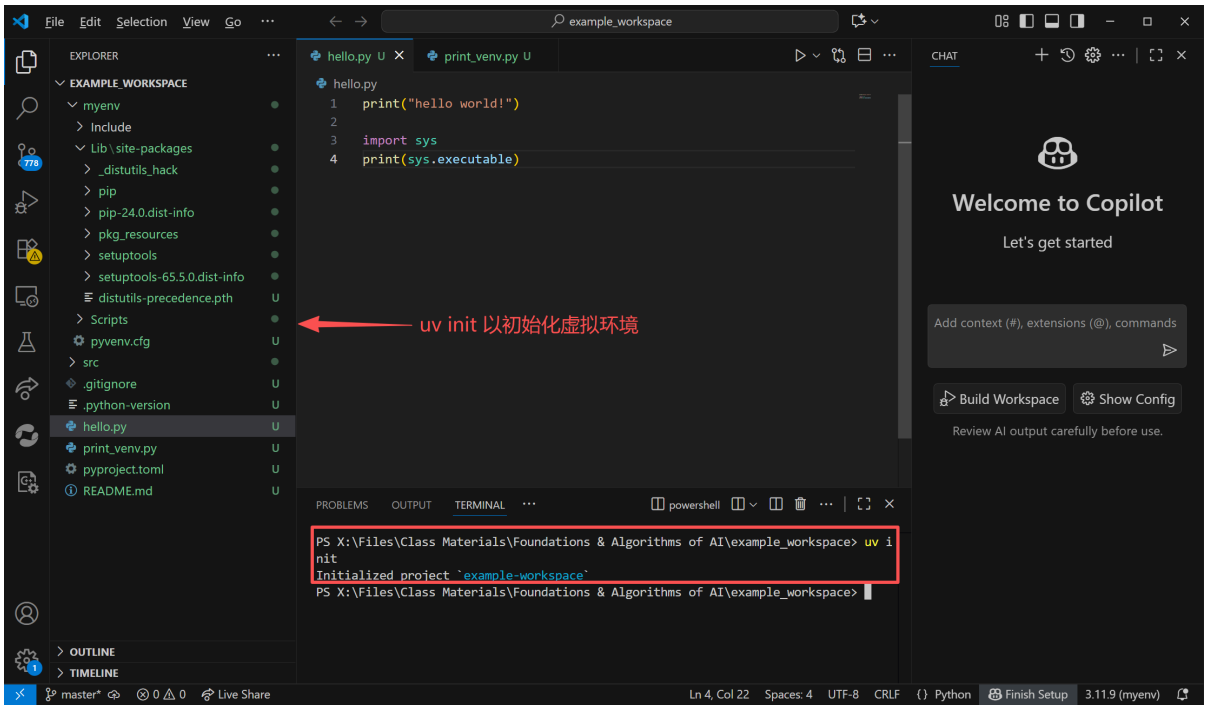
```
winget install uv
```

随后重启 VSCode 以更新环境变量。安装完成后，可输入

```
uv --version
```

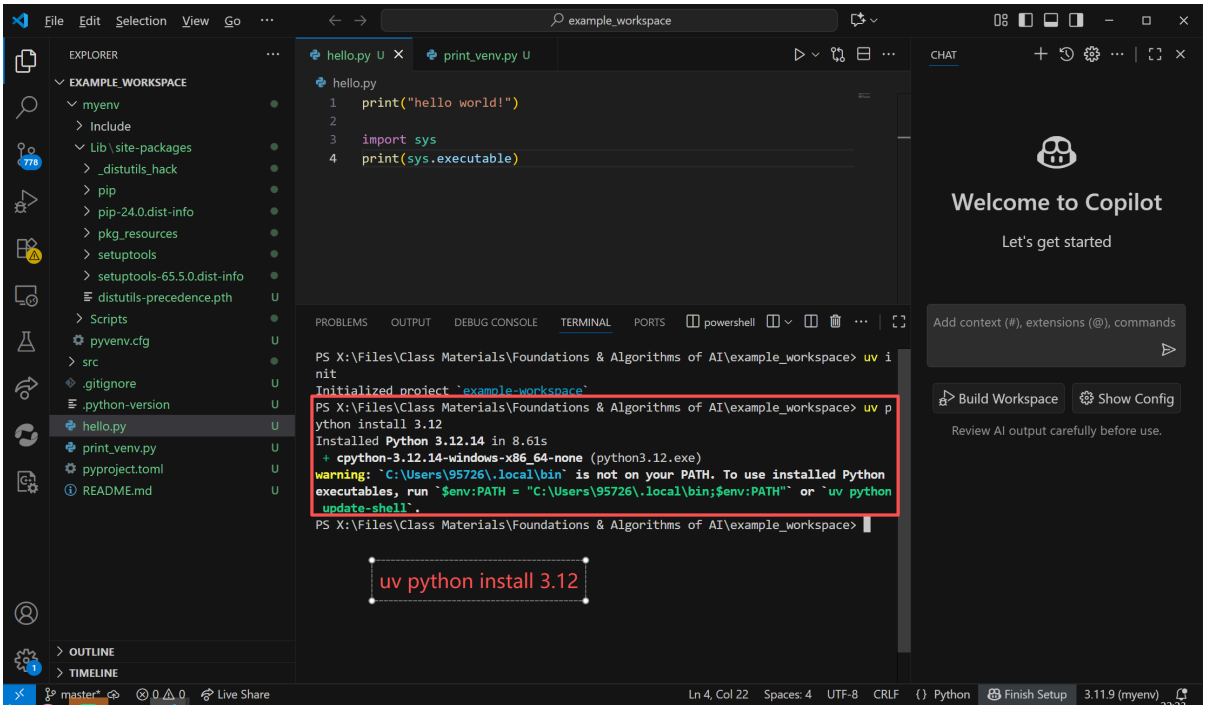
以检查 uv 是否被正确安装。使用时，先初始化虚拟环境

```
# 若不指定环境名，它将默认以 venv 作为环境名称：
uv init
```



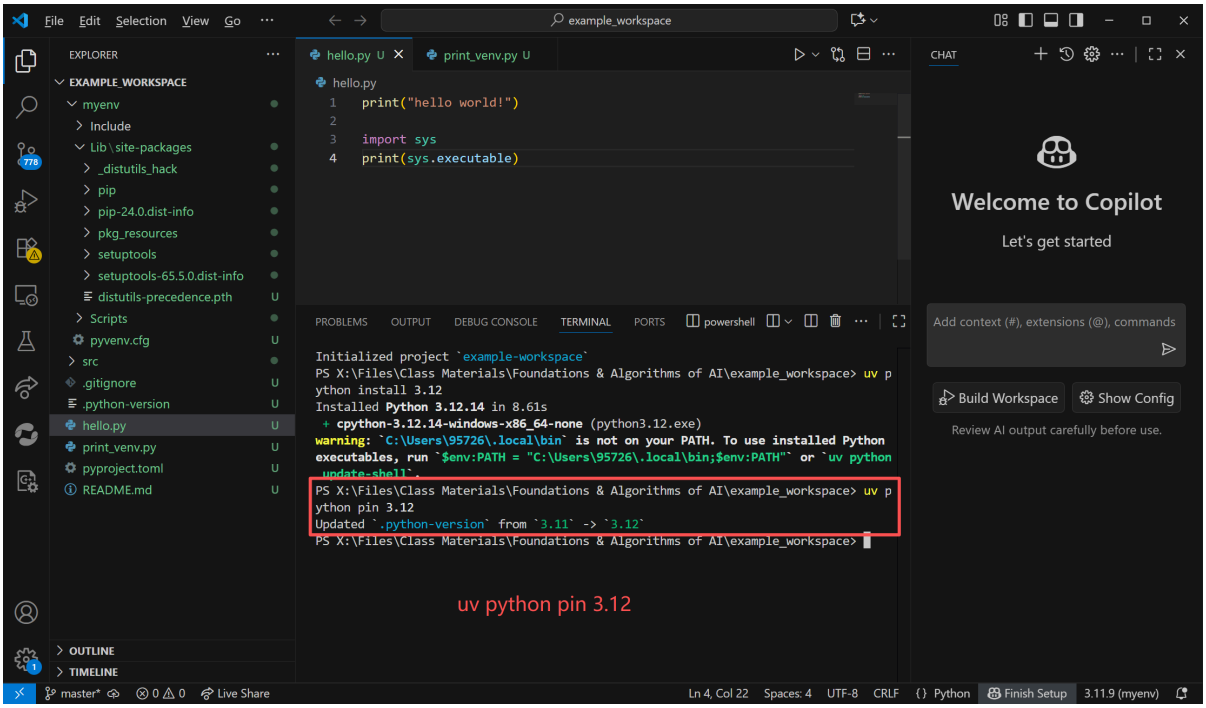
接着可以指定 Python 的版本：

```
uv python install 3.12
```



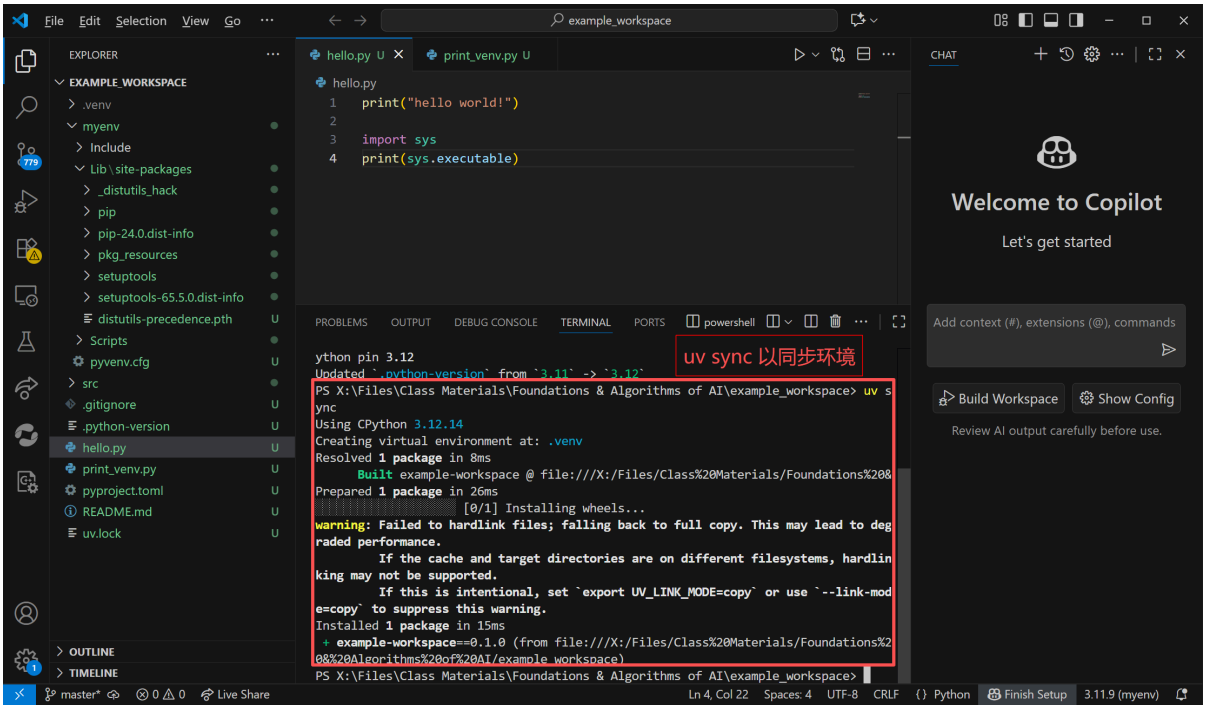
将当前环境设定为刚才安装的 Python 3.12：

```
uv python pin 3.12
```



最后将这些设置同步到虚拟环境中

```
uv sync
```

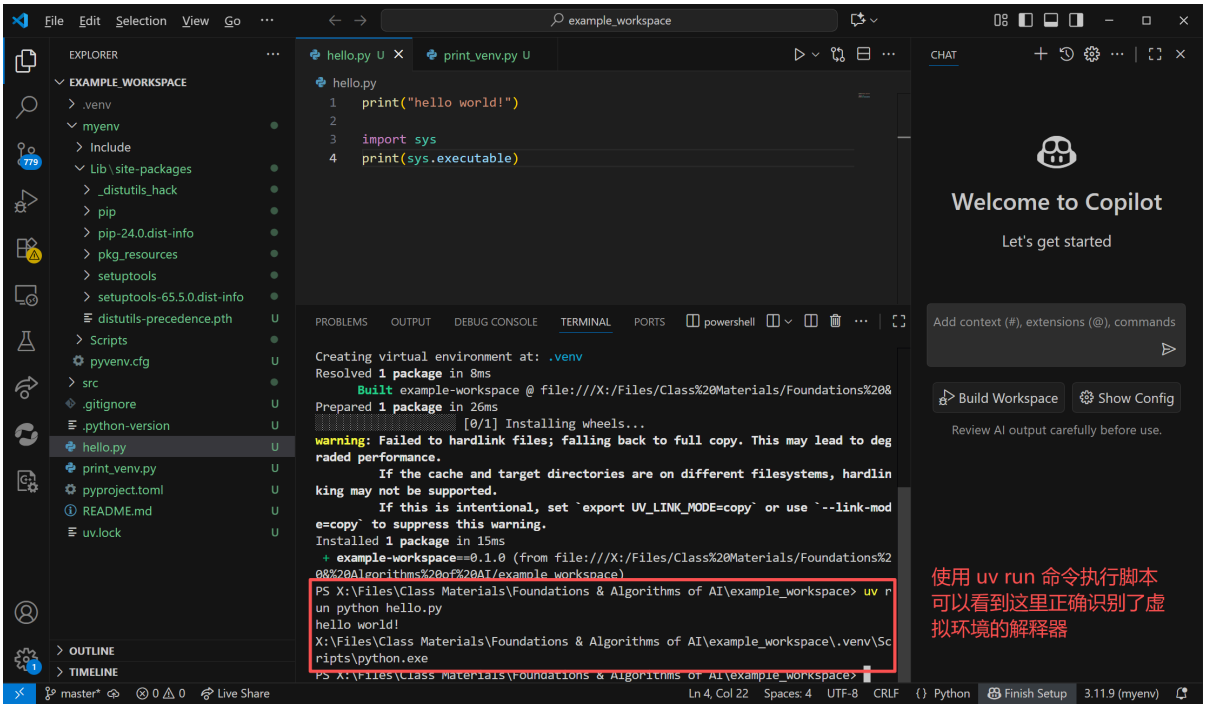


若要在环境中安装包，使用 `uv add` 命令，环境会自动同步：

```
# 以 numpy 安装为例
uv add numpy
```

也可以显式调用 `venv` 去执行脚本，它会自动识别当前虚拟环境下的解释器：

```
uv run python hello.py
```



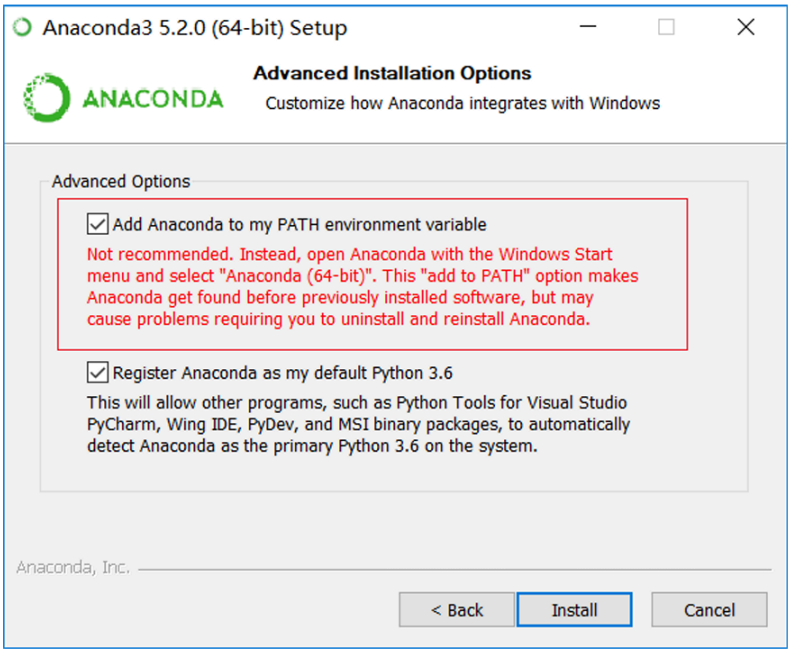
若要让 VSCode 同样识别该解释器，按照 venv 中的教程选择 Python: Select Interpreter 即可。

Anaconda 的安装与配置

进入清华大学开源软件镜像站中 Anaconda 的镜像页面：
https://mirrors.tuna.tsinghua.edu.cn/anaconda/archive/ 选择合适的 Anaconda 版本



安装时建议不要改动 Conda 的安装路径（尤其是使用 Pycharm 开发的同学）。此外，需要勾选 "将 Anaconda 加入到 PATH 环境变量中"



安装程序这里提示说不建议，是因为现在的 Anaconda 已经内置了自己的终端乃至是 GUI 管理界面。但如果不考虑 Anaconda 之间的版本冲突问题（换句话说你如果要安装好几个 conda），将其加入到环境变量实际上是没有问题的。

安装完成后，可以自行打开终端测试 conda 是否安装完成（Win+R 后输入 cmd 以打开终端）



现在使用 conda 创建虚拟环境：

```
conda create -n myenv python=3.9
```

激活虚拟环境：

```
conda activate myenv
```

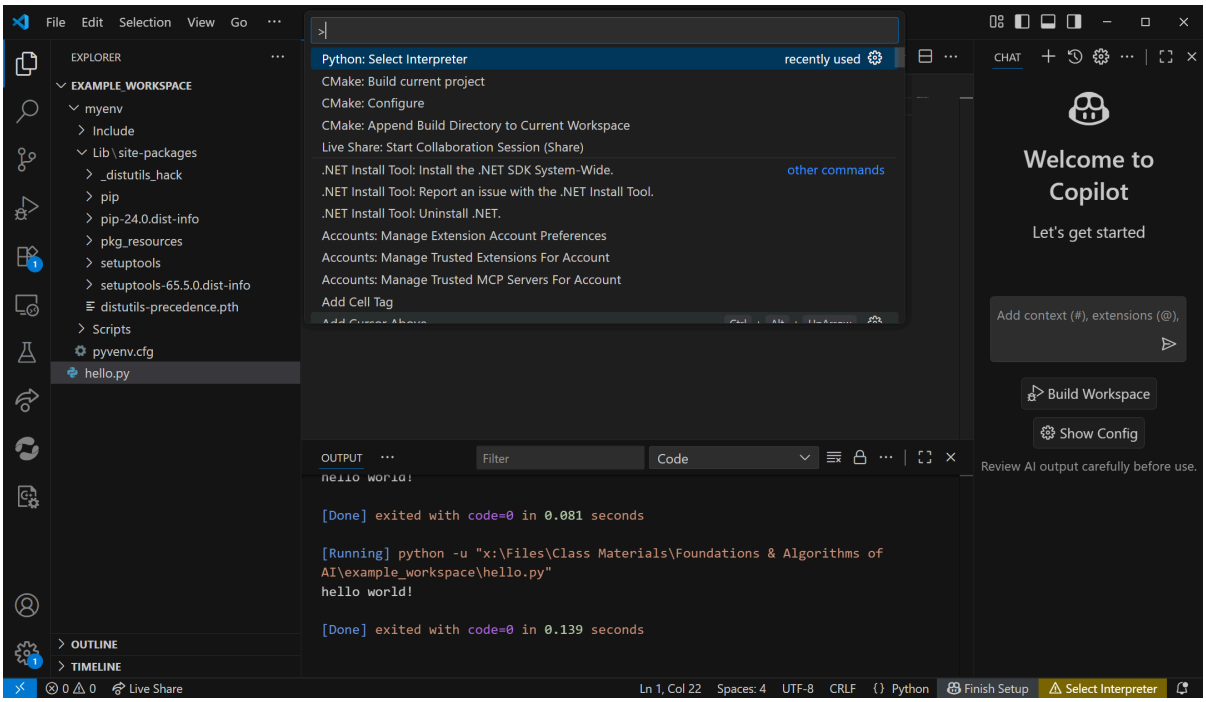
当终端出现 `(myenv)` 的前缀时，说明虚拟环境已激活。若出现网络问题，则一般需要为 conda 配置国内镜像源。具体可查阅网络。

实际上同学们的综合实验将会在超算平台上使用，那里已经配置好了 conda，且以后同学们不大可能会在自己的个人电脑上进行 python 开发，所以其实并不刚需 conda。这也是为什么文档提供了 venv 的虚拟环境配置方式。

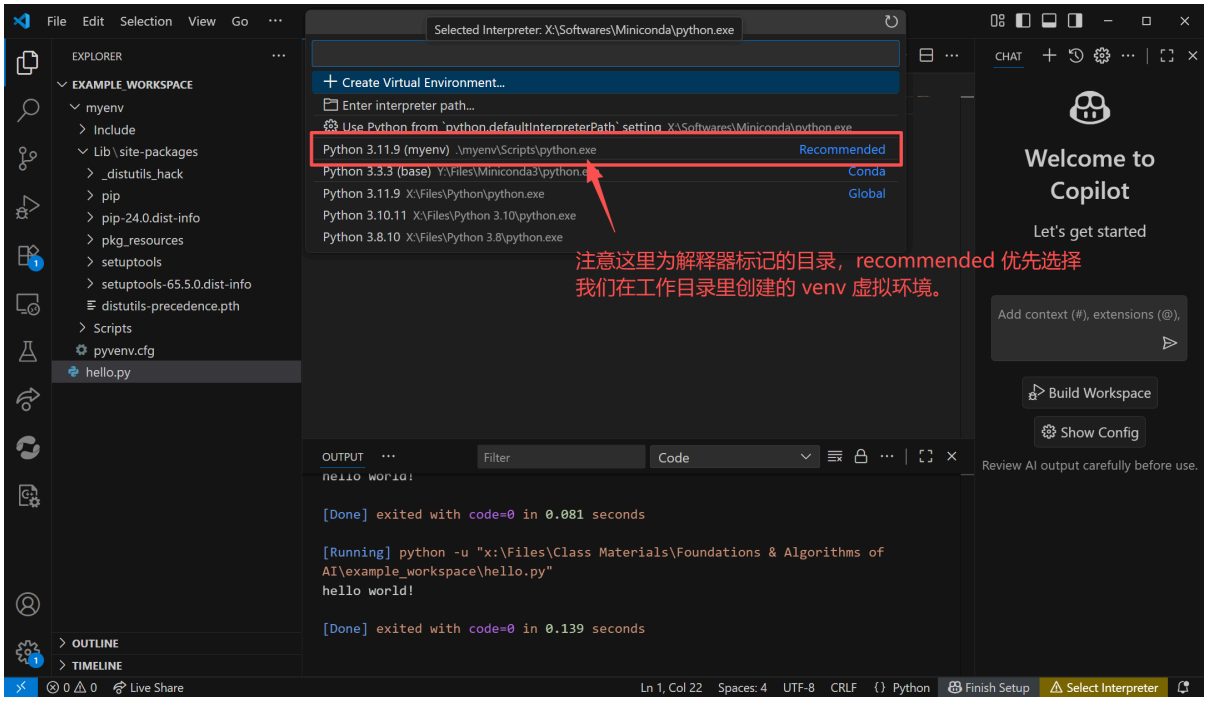
VSCode 选择解释器

若要使用方法一（即插件自带的执行按钮）来运行处于虚拟环境下的脚本，这会因为插件会在自己创建的终端上执行解释器而出现报错。这时需要为 VSCode 指定插件所使用的解释器。

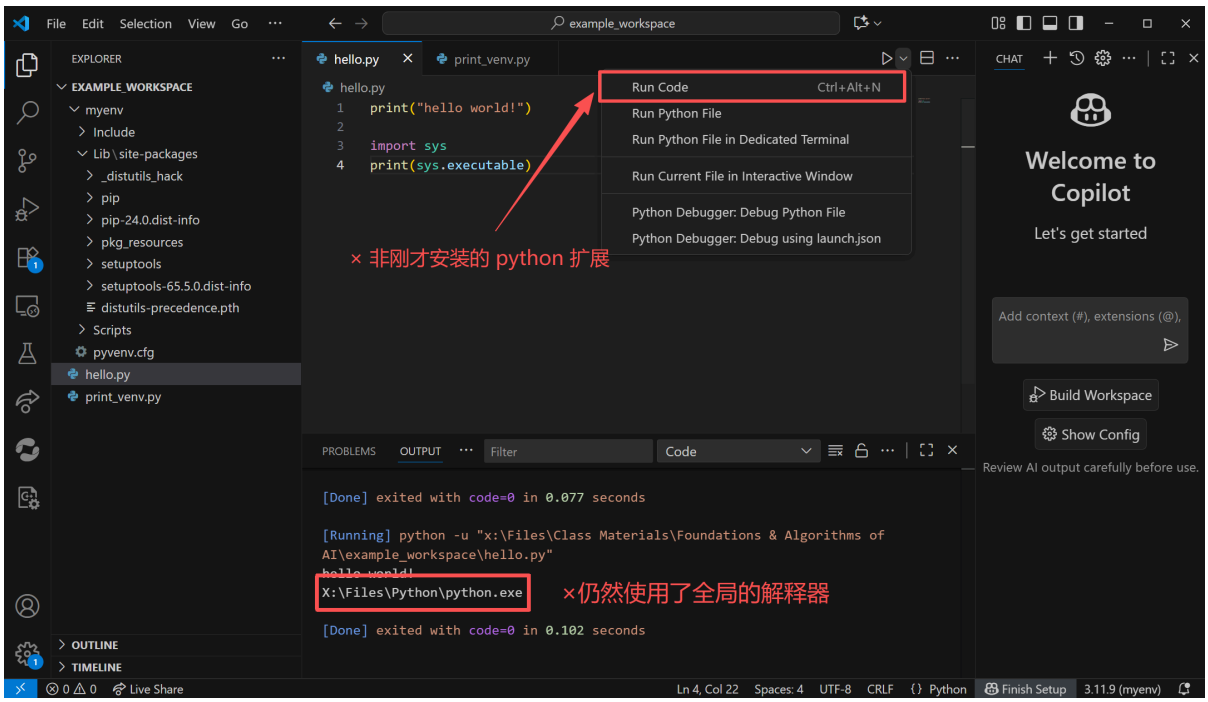
在 VSCode 中，按下 `Ctrl + Shift + P` ，可以呼出 VSCode 的指令列表。在其中搜索并选择 '> Python: Select Interpreter'

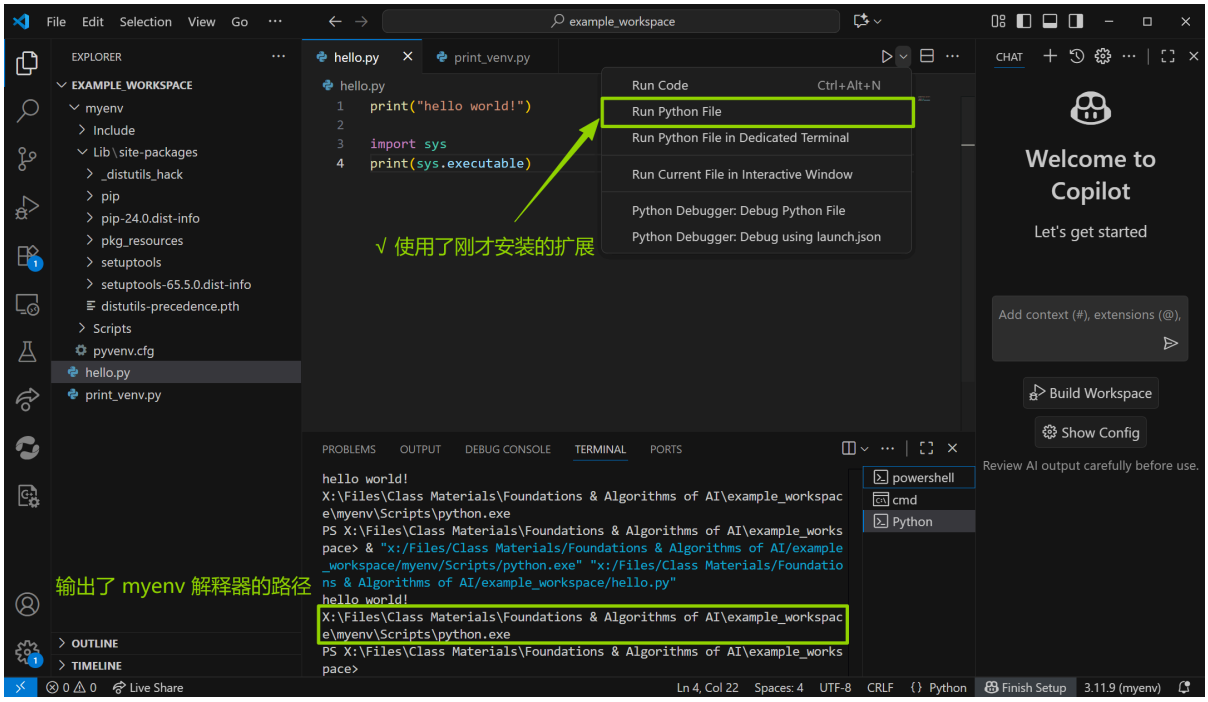


然后在其中选择我们刚刚创建的 `myenv` 下的解释器：



此时即可使用虚拟环境的解释器执行代码。但需要注意不应使用执行选项中的 `Run Code`，而应该使用 `Run Python File`。前者是由插件 `Code Runner – formulahendry.code-runner` 启动的，并不是我们安装的 Python 插件。





此外还有一点需要注意： `myenv` 中的解释器仅仅是全局解释器的软链接（Linux 下）或复制（Windows 下），它不能离开全局 Python 单独作为完整的解释器使用。换句话说，若删除了全局的 Python， `myenv` 中的解释器也将无法执行。

01 实验目标与任务

1 计算等额本息还款金额

目标是考查单值变量的定义与基本计算。

等额本息是一种分期偿还贷款的方式，即借款人每月按相等的金额偿还贷款本息，每月还款金额 P 可根据贷款总额 A、年利率 r 和贷款月数 n 计算得到，公式为

$$P = \frac{\frac{r}{12}A}{1-\left(1+\frac{r}{12}\right)^{-n}}$$

计算当贷款金额为 1000000，贷款时间为 30 年，年利率分别为 4%、5% 和 6% 时的每月还款金额和还款总额。输出结果在小数点以后保留两位有效数字。编写代码计算结果后，与答案进行比对。

答案：

```
(4774.15, 1718695.06)
(5368.22, 1932557.84)
(5995.51, 2158381.89)
```

2 计算三角形的内角

目标是考查数学库（math）引入的高阶运算与函数调用、布尔类型单值的使用。

定义三个变量 “a=3; b=6; c=7” 表示一个三角形的三个边的长度，使用公式

$$a^2 = b^2 + c^2 - 2bc\cos\alpha$$

$$b^2 = a^2 + c^2 - 2ac\cos\beta$$

$$c^2 = a^2 + b^2 - 2ab\cos\gamma$$

分别计算三个内角 (α, β, γ) 的度数，然后检验等式

$$\alpha + \beta + \gamma = 180$$

是否成立。

在该实验中，需要使用 `import` 语句引入数学库，框架代码如下：

```python
# calculate_inner_angle.py
# 导入 math 库
import math

# Your implementation here below...
```

计算完三个内角后，可与如下答案比较：

答案：

```
25.208765296758365 58.41186449479884 96.37937020844281
```

接着需要使用布尔变量判断三个角内角和是否为 180°。**注意，本实验要求不可使用 if 分支结构**。（现在暂时不许）

3 内积与二阶矩阵求逆

目标是考查列表元素的表示与计算。

在深度学习等领域，计算机存储并表示一个矩阵或张量的方式常常是通过嵌套列表或数组。从直觉来看，一个一维列表可以用于表示一个向量，而一个二维列表即可用于表示任意规模的矩阵，其中一个维度对应其行，另一个维度对应其列。例如，一个行向量和三维矩阵

$$\mathbf{x} = [1, 1, 4, 5, 1, 4], \quad A = \begin{bmatrix} 1 & 9 & 1 \\ 9 & 1 & 8 \\ 1 & 0 & 0 \end{bmatrix}$$

可以有如下的列表表示

```python
x = [1, 1, 4, 5, 1, 4]

A = [
    [1, 9, 1],
    [9, 1, 8],
    [1, 0, 0]
]
```

两个向量的内积可以表示为

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \cdot \mathbf{y} = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n.$$

对于一个二阶矩阵

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

我们有如下简易的求逆公式

$$A^{-1} = \frac{1}{\det A} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

该实验的任务是计算三维向量的内积和一个二维矩阵的逆。注意，**本实验不可以使用 for 循环**（现在暂时不许），框架代码和数据已经给出：

```
# calculate_innerprod_and_inverse.py

x = [2, 3, 3]
y = [6, 6, 6]
A = [
    [1, 2],
    [3, 4]
]

# 1: Calculate the inner product <x,y>
inner_prod =

# 2: Calculate the inverse matrix
A_inv =

'''
You can double-check whether A \cdot A_inv gives you the identity matrix,
i.e. only the diagonal elements of result being 1 and others being 0.
That will be another easy work though, but it's optional. ;)
'''
```

一个澄清：可以注意到实际上内积涉及了向量的转置，而在本次实验中我们不需要考虑这一点（你可以思考列向量是如何表示的，而表示之后你就会发现有多么不方便计算）。这实际上导致两个向量的内积变成了逐元素乘积。上述表示只是一种方便理解和计算的方式，在实际的向量计算库如 `numpy` 中，其对张量的表示会封装为 `np.array` 这样的类，在 `torch` 中则封装成了 `tensor` 类。

4 列表与字符串操作

目标是考查列表与字符串操作。

已经给定了列表和字符串的操作脚本 `list_str_op.py` ，要求首先根据自己的思考预测执行结果，然后运行脚本看最终结果，填写其中的注释。

02 验收内容

1~3 实验要求提供源代码与终端执行结果。实验 4 要求提供填空后的源代码，并注意思考有哪些地方执行结果与自己的预想有出入。